Stem Agent Write-up

Bogdan Purdea

1 Scope and Possible Task Domains

The chosen scope is software and agent system engineering, that cover task domains such as software architecture, prototyping, implementation, code refactoring, database management, agent orchestration, system evaluations design, deployment, security and others.

2 Approach and Evolution

2.1 Initial Concept

A conceptual idea I had initially was an agent represented as a state machine provided with a minimal set of tools and skills essential to software and agent system design, such as file system access, compilers, testing frameworks, software design and implementation best practices. Maybe even provide it with a factual knowledge base on the subject through Retrieval-Augmented Generation. The agent would use those tools and provided factual context to generate and execute code that modifies its own state, searching for best practices, creating new tools, and extending its behavior. It would build itself into a mature state, creating reusable agent states and its own evaluation pipelines to decide when to stop evolving. A possible risk with this approach is the stem agent evolving into a failing state after multiple subsequent hallucinations. Correctness checks at every step in the pipeline and a rollback mechanism to previous states could mitigate this risk.

2.2 Research

Considering the complexity of such a system and my own somewhat limited experience in AI agent development, I moved toward researching existing implementations of the stem agent idea.

I discovered the recent STEM Agent paper submitted on 22 March this year. While its scope is more general, it shares the same biological inspiration. The paper describes a five-layer architecture and an eight-phase cognitive pipeline algorithm: Perception → Adaptation → Skill Match → Reasoning → Execution → Formatting → Learning. It also implements concepts like crystallization for maturity and apoptosis for persistent failure.

2.3 Simplified Architecture

Recognizing that the paper's implementation is a highly ambitious work-in-progress, I decided to start small, building something simpler that might have potential to get the job done. Taking some inspiration from the previously mentioned research paper, I simplified the pipeline to four phases: Perception → Adaptation → Planning → Execution. I moved from a broad, probabilistic implementation to a more deterministic one that is less reliant on LLM reasoning to go perfectly. I chose this approach to produce a valid prototype in a short timespan, as iterating from a simpler system is more efficient than attempting to validate a complex one from the start.

3 Development Tool Selection

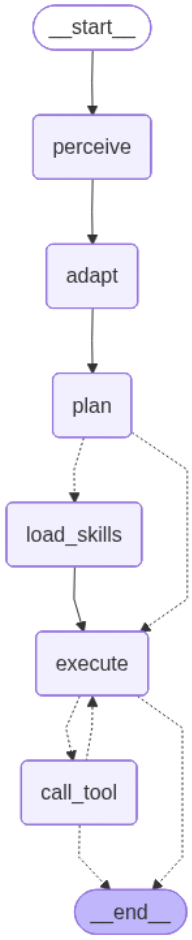
I chose the LangChain Agent Stack (LangGraph and LangSmith Studio) for several reasons:

- LangGraph allows custom agent orchestration via a State Graph, where nodes update the state and edges drive the execution flow. This feature could provide a way for a stem graph to rewrite it's own State Graph.

- LangSmith Studio, provides a graph visualization feature which, in the case of a self adapting stem agent, could provide transparency and decision feedback to human users. It also provides evaluation and tracing capabilities within the same environment, simplifying the agent development process.
- LangGraph checkpoints save graph state so runs can be resumed from saved snapshots. This could help the stem agent recover in case it evolves to a continuously failing state due to multiple hallucinations for example.
- My personal interest in learning custom agent orchestration at a lower level rather than relying on pre-built abstractions.

4 System Architecture

The agent uses a LangGraph state machine to differentiate itself, attempting to transition from a general-purpose agent into a specialized agent tailored for the perceived task domain. It perceives it's task, adapts to it by setting some behavior parameters, plans accordingly, then it executes the plan. In this specific case the adaptation is deterministic, it simply maps a reasoning strategy to the perceived complexity of the task. The perception, planning and execution is done using LLM calls.



4.1 Cognitive Pipeline

The agent transitions through the following nodes in the state graph:

- **Perceive:** The agent identifies from the incoming query a structured signal, extracting domain, complexity, intent, and context hints (keywords).
- **Adapt:** A deterministic mapping translates complexity into a specific reasoning strategy (currently ReAct is employed for all tasks, Reflexion remains to be implemented for complex tasks), sets the maximum iterations to avoid infinite loops at execution and sets a fitting persona for the perceived domain.
- **Plan:** Provided the domain, intent and reasoning method it produces a set of ordered plan steps and manifests for tools and skills.
- **Skill Loading:** When skills are selected in the planning phase, their full `SKILL.md` contents are injected into the execution context.
- **Execute:** The model is called using the chosen persona and plan, with only the planned tools bound for that run.
- **Tool calling:** Invokes the tool and returns the result to the execution node or breaks the loop if the consecutive failure threshold is reached.

4.2 Safeguards and Integrity

To prevent infinite failure loops, a circuit breaker exists. This maintain a counter for consecutive failures. If the threshold is reached, execution halts and returns a circuit-breaker result, ensuring the agent rebuilds its approach rather than breaking the system. While I understand the importance of guardrails, I did not find the time to build comprehensive ones in this iteration of the project.

5 Evaluation and Results

I chose a basic ReAct agent as the baseline since even for simple tasks, agents can benefit from tool usage. It has access to the same tools, but no access to skills.

The agent was provided with tools based capabilities (e.g.recursive file search, file reading and web search). Skills on LangChain and LangGraph agent architecture were also provided. Since this was my first time using LangSmith for building datasets and evaluations, I created a dataset of five software engineering and LangGraph based agent

orchestration tasks ranging from simple to complex and one simple task focusing on tool usage capability. This is a very small sample size, but it's main purpose it to test the evaluation pipelines. Evaluation on real SE benchmarks and datasets would be a next step.

5.1 Experimental Metrics

The agents were evaluated on the following metrics:

- **Correctness:** LLM-as-judge scoring via OpenEvals.
- **Differentiation:** Accuracy of perceived vs. expected complexity.
- **Tool Efficiency:** Tool precision and recall against an expected standard toolset.

5.2 Outcome Analysis

Using an llm-as-a-judge with a generic correctness prompt from openeval, the stem agents correctness score kept oscillating from one evaluation run to another. Once tool and skill access information was introduced in the correctness prompt as additional context, the stem agent achieved a high correctness score, performing better than the baseline ReAct agent, which struggled to provide entirely correct responses.

The performance of the stem agent was the same as the baseline for tool precision, with a slightly better tool recall score.

However, it had a higher token generation cost, making it a questionable choice over the more token efficient baseline for these specific tasks.

The results suggest more complex input tasks are required to possibly justify the adoption of this stem agent over a simple ReAct agent. I also believe a more comprehensive array of tools and skills could increase the effectiveness of this pipeline on complex tasks.

The experiments are made public here: STEM Agent Benchmark.

6 Future Work

Given more time, I would improve this project by:

- Adding the reflection loop for complex tasks
- Implementing thorough code validation and error handling
- Increase unit and integration testing coverage
- Building a more comprehensive dataset using existing SE benchmarks
- Enforcing advanced safeguarding and guardrail strategies at each step of the pipeline.
- Refactoring the code toward a more object-oriented approach to support better scalability and maintainability.
- Complementing evaluation metrics with more relevant ones.
- Leveraging State Graph checkpoints to reset the agent to a working state in case of persistent failure
- Extending the range of discoverable skills through trusted MCP servers and skill directories like skills.sh.
- Implementing skill creation and storage for successfully completed tasks to make them reusable—completing the maturation loop.

I would also like to attempt the more probabilistic architectural approach, building an agent capable of adapting itself to the task domain by modifying it's own State Graph, searching tools and skills and evaluating it's own performance.

7 Personal outcome

I chose to make use of tools like LangGraph and LangSmith because they seemed a great fit for the stem agent concept, which requires deeper orchestration than conventional agent architectures. This process allowed me to advance my understanding of custom agent orchestration, an area I am actively pursuing in my free time, while teaching me how to build datasets and evaluations within the LangSmith ecosystem.

This project was a very exciting task for me. Although my simplified implementation strayed from the original probabilistic idea due to my current lack of practical experience in the field, it provided a valuable opportunity to further my knowledge in the field of agent engineering. Regardless of the outcome, this has been a significant learning experience. Thank you for the opportunity and for taking the time to read this write-up.